

BANCO DE DADOS

DO CONCEITO À PRÁTICA



Curso Técnico em Desenvolvimento de Sistemas

- **Disciplina:** Banco de Dados **Carga Horária:** 125 horas

APRESENTAÇÃO

Bem-vindo à disciplina de Banco de Dados.

Este material foi desenvolvido especialmente para você, estudante do Curso Técnico em Desenvolvimento de Sistemas, com uma linguagem clara, exemplos práticos e progressão didática cuidadosamente planejada. Cada capítulo foi construído partindo do simples para o complexo, conectando a teoria ao que você vai encontrar no mercado de trabalho.

Você já chegou aqui com conhecimentos em lógica de programação, modelagem de sistemas e fundamentos de tecnologia da informação. Agora vai descobrir como os dados são organizados, armazenados, protegidos e consultados em sistemas reais — desde o aplicativo de delivery que você usa no celular até os sistemas bancários que movimentam trilhões de reais por dia.

Ao longo desta apostila você vai construir bancos de dados do zero, escrever consultas SQL que respondem perguntas de negócio reais, modelar sistemas completos, proteger dados sensíveis e entender como funciona o mundo dos dados em escala corporativa.

Para aproveitar este material ao máximo, você vai precisar de um computador com MySQL 8.x instalado, o MySQL Workbench e o brModelo. Todas as atividades foram testadas e funcionam nesses ambientes.

SUMÁRIO

Sumário

.....	1
Curso Técnico em Desenvolvimento de Sistemas.....	2
1.1 Dado, Informação e Conhecimento	5
1.2 O Problema de Gerenciar Dados sem um Banco de Dados	6
1.3 O que é um Banco de Dados.....	7
1.4 História e Evolução dos Bancos de Dados	8
1.5 Tipos de Banco de Dados.....	9
1.6 Estudo de Caso: A Migração da MercadoTech.....	10
1.7 Exercícios do Capítulo 1	10
2.1 O que é um SGBD.....	12
2.2 Arquitetura de Três Camadas (ANSI/SPARC)	12
2.3 Principais SGBDs do Mercado.....	13
2.4 Instalação do MySQL e MySQL Workbench	13
2.5 Exercícios do Capítulo 2	14
3.1 Por que Modelar antes de Implementar	15
3.2 Entidades, Atributos e Relacionamentos	15
3.3 Tipos de Atributos.....	16
3.4 Cardinalidade	16
3.5 Entidade Fraca	17
3.6 Generalização e Especialização	17
3.7 Notações do DER.....	17
3.8 DER Completo: Sistema de Clínica Médica	18
3.9 Exercícios do Capítulo 3	18
4.1 Regras de Transformação do DER para o Modelo Lógico	19
4.2 Tipos de Dados no MySQL.....	20
Tipos Inteiros	20
Tipos Decimais	20
Tipos de Texto	21
Tipos de Data.....	21
4.3 Constraints	22
4.4 Script DDL Completo: Clínica Saúde Plena	23
4.5 Exercícios do Capítulo 4	25
5.1 O Problema da Redundância.....	26
5.2 Dependência Funcional	26
5.3 Primeira Forma Normal — 1FN	27
5.4 Segunda Forma Normal — 2FN	27

5.5 Terceira Forma Normal — 3FN.....	27
5.6 Processo Completo de Normalização — Exemplo.....	28
5.7 Desnormalização Estratégica	28
5.8 Exercícios do Capítulo 5	29
6.1 Categorias do SQL	29
6.2 DDL — Criando Estruturas	30
6.3 DML — Manipulando Dados	31
6.4 Transações — ACID na Prática	32
6.5 Exercícios do Capítulo 6	33
7.1 Estrutura Completa do SELECT	33
7.2 Filtros e Funções Essenciais	34
7.3 JOINS em Profundidade.....	35
7.4 Subqueries e CTEs.....	36
7.5 Window Functions	37
7.6 Exercícios do Capítulo 7	37
8.1 Views.....	38
8.2 Stored Procedures.....	39
8.3 Functions.....	41
8.4 Triggers	42
8.5 Exercícios do Capítulo 8	43
9.1 Principais Ameaças	44
9.2 SQL Injection — Ataque e Defesa	44
9.3 Gerenciamento de Usuários e Privilégios	45
9.4 LGPD e Banco de Dados.....	46
9.5 Backup e Restauração	46
9.6 Performance com EXPLAIN	47
9.7 Exercícios do Capítulo 9	48

CAPÍTULO 1 — FUNDAMENTOS DE BANCO DE DADOS

1.1 Dado, Informação e Conhecimento

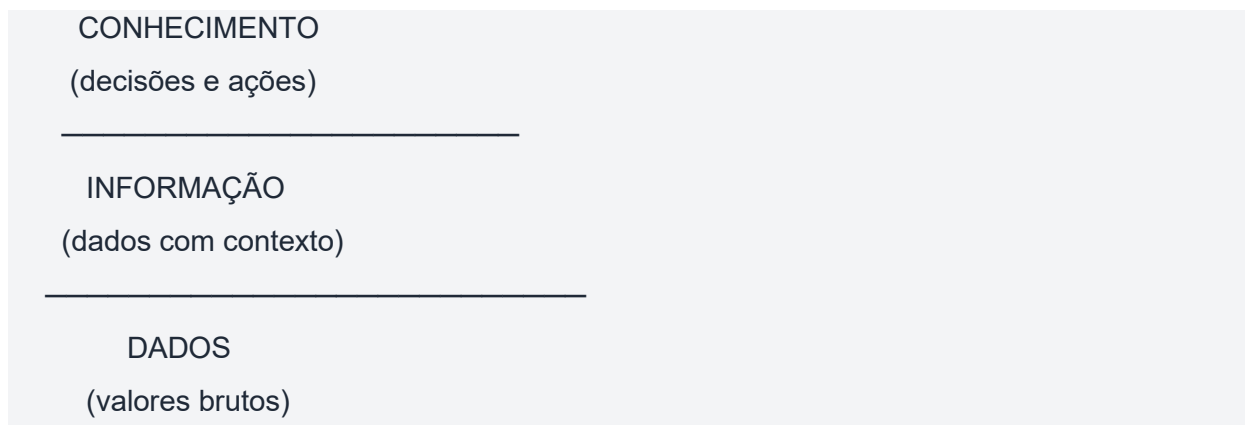
Antes de entender o que é um banco de dados, precisamos distinguir três conceitos que parecem sinônimos mas são completamente diferentes no contexto da tecnologia da informação: dado, informação e conhecimento.

Um **dado** é um valor bruto, isolado, sem contexto. O número 28, a letra A, o texto "Curitiba" e o valor 1500.00 são dados. Sozinhos, eles não dizem nada. Você olha para o número 28 e se pergunta: 28 o quê? Temperatura? Idade? Número de produtos? Sem contexto, um dado não tem utilidade prática.

A **informação** surge quando damos contexto ao dado. Quando dizemos "a temperatura em Curitiba hoje é de 28 graus Celsius", transformamos o dado bruto "28" em uma informação útil. A informação tem significado, contexto e relevância para quem a recebe.

O **conhecimento** vai além da informação. Ele é construído quando combinamos informações com experiência e raciocínio para tomar decisões. Saber que "em Curitiba, quando a temperatura ultrapassa 28°C em fevereiro, costuma chover à tarde" é conhecimento — ele permite agir de forma mais inteligente.

Pirâmide do Conhecimento:



Exemplo prático:

Um sistema de vendas armazena o dado "1500.00" na coluna "valor_venda". Esse dado, combinado com a data, o vendedor e o produto, forma uma informação: "O vendedor João vendeu o produto X por R\$ 1.500,00 no dia 15/03/2024". Quando analisamos essa informação junto com outras vendas, podemos construir conhecimento: "João é o vendedor mais eficiente às quintas-feiras".

1.2 O Problema de Gerenciar Dados sem um Banco de Dados

Antes dos bancos de dados, as empresas armazenavam informações em fichas de papel, depois em arquivos de texto e planilhas eletrônicas. Esse modelo funciona para volumes pequenos, mas rapidamente se torna inviável à medida que o negócio cresce.

Imagine uma empresa com 10 mil clientes gerenciada em planilhas Excel. O vendedor A mantém sua lista de clientes em uma planilha. O vendedor B mantém a dele em outra. Quando um cliente muda de endereço, quantas planilhas precisam ser atualizadas?

Esse cenário ilustra os três principais problemas do gerenciamento de dados sem um banco de dados adequado:

- **Redundância:** o mesmo dado está armazenado em múltiplos lugares, desperdiçando espaço e criando possibilidade de inconsistência.
- **Inconsistência:** quando os dados estão duplicados, é quase inevitável que fiquem diferentes entre si.
- **Dificuldade de acesso concorrente:** planilhas não foram projetadas para acesso simultâneo por múltiplos usuários.

1.3 O que é um Banco de Dados

Um banco de dados é uma coleção organizada de dados relacionados entre si, armazenados de forma estruturada, com redundância controlada, que pode ser acessada por múltiplos usuários simultaneamente com garantia de consistência e segurança.

As principais características de um banco de dados de qualidade são:

- **Persistência:** os dados sobrevivem ao encerramento do programa.
- **Compartilhamento:** múltiplos usuários podem acessar os mesmos dados simultaneamente.
- **Integridade:** regras automáticas garantem que os dados sejam válidos.
- **Independência de dados:** a forma interna de armazenamento é separada da forma como os programas acessam os dados.
- **Segurança:** controle preciso de quem pode ver, inserir, alterar ou excluir cada dado.
- **Controle de redundância:** evita que o mesmo dado seja armazenado múltiplas vezes desnecessariamente.

1.4 História e Evolução dos Bancos de Dados

- **Década de 1960:** processamento por arquivos sequenciais em fita magnética. Surgem os primeiros SGBDs com modelos hierárquicos (IMS da IBM) e de rede.
- **1970:** Edgar F. Codd, da IBM, publica o artigo "A Relational Model of Data for Large Shared Data Banks", propondo organizar dados em tabelas bidimensionais com álgebra relacional.
- **1974:** Donald Chamberlin e Raymond Boyce desenvolvem a linguagem SEQUEL, que se tornaria o SQL.
- **1979–1996:** surgimento dos primeiros SGBDs comerciais: Oracle (1979), DB2 (1980), SQL Server (1989), MySQL (1995), PostgreSQL (1996).
- **Anos 2000:** a explosão da internet gera volumes de dados sem precedentes. Surgem limitações dos bancos relacionais em escala massiva.
- **2009 em diante:** movimento NoSQL com MongoDB, Cassandra, Redis, Neo4j. A combinação de bancos relacionais e NoSQL passa a ser o padrão em sistemas modernos.

1.5 Tipos de Banco de Dados

Tipo	Características	Exemplos
Relacional	Tabelas, SQL, ACID, relacionamentos com JOINS	MySQL, PostgreSQL, Oracle, SQL Server
NoSQL — Documentos	Documentos JSON flexíveis, schema variável	MongoDB, CouchDB, Firebase
NoSQL — Chave-Valor	Estrutura simples, extremamente rápido	Redis, Memcached, DynamoDB
NoSQL — Colunar	Otimizado para consultas em colunas específicas	Cassandra, HBase
NoSQL — Grafos	Dados interconectados em rede	Neo4j, Amazon Neptune
NewSQL	Combina SQL com escalabilidade horizontal	Google Spanner, CockroachDB
Em Memória	Armazenamento em RAM, latência mínima	Redis, H2

1.6 Estudo de Caso: A Migração da MercadoTech

A empresa MercadoTech é uma distribuidora de produtos eletrônicos fundada em 2018. Nos primeiros anos, o controle era feito em planilhas Excel. Em 2021, com 500 clientes ativos e 3.000 pedidos por mês, os problemas se tornaram críticos: clientes cadastrados duas vezes, preços desatualizados em algumas planilhas, relatórios inconsistentes.

O ponto de ruptura veio quando um vendedor excluiu acidentalmente 200 linhas de pedidos sem backup recente, causando perda de dados de dois dias de operação.

Após a migração para o MySQL: dados com fonte única da verdade, atualizações propagadas automaticamente, controle de acesso por perfil, backups automáticos a cada hora e relatórios sempre consistentes.

1.7 Exercícios do Capítulo 1

Exercício 1 — Classifique cada item como dado, informação ou conhecimento, justificando sua resposta:

- a) O número 42
- b) "O produto X foi o mais vendido em dezembro de 2024 com 1.200 unidades"
- c) "Clientes que compram produtos da categoria A tendem a comprar da categoria B no mês seguinte"
- d) A string "joao.silva@email.com"
- e) "João Silva realizou 15 compras nos últimos 30 dias com ticket médio de R\$ 250,00"

Exercício 2 — Uma escola gerencia notas em planilhas Excel, uma por turma. Identifique 5 problemas que esse modelo pode causar e proponha como um banco de dados resolveria cada um.

Exercício 3 — Para cada cenário abaixo, indique se usaria banco relacional, NoSQL ou ambos, justificando:

- a) Sistema de controle de estoque de supermercado
- b) Rede social com 50 milhões de usuários
- c) Sistema de monitoramento de 10.000 sensores IoT em tempo real
- d) Sistema de gestão de RH com 500 funcionários
- e) Plataforma de streaming com recomendações personalizadas

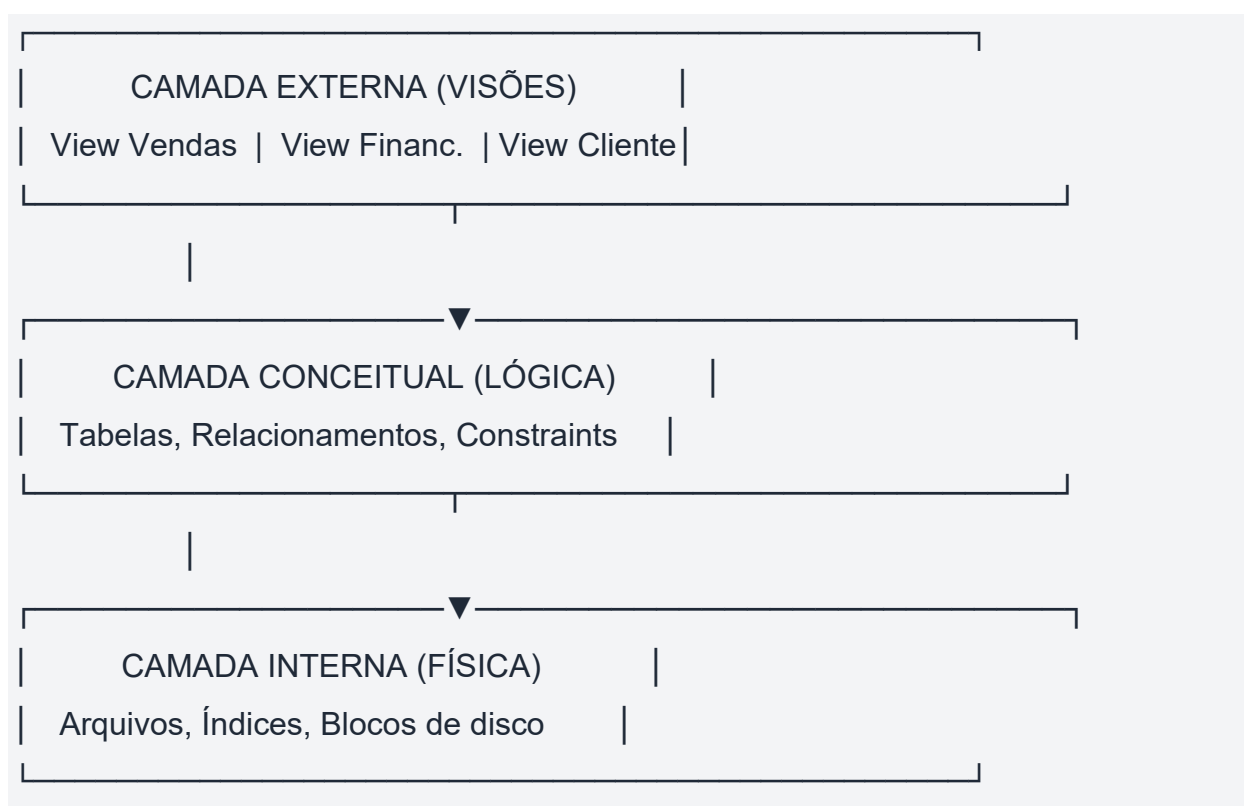
CAPÍTULO 2 — ARQUITETURA E SISTEMAS GERENCIADORES DE BANCO DE DADOS

2.1 O que é um SGBD

Um Sistema Gerenciador de Banco de Dados (SGBD) é o software responsável por criar, manter, controlar e fornecer acesso aos bancos de dados. É o intermediário entre os programas e os dados.

Pense no SGBD como o gerente de um grande arquivo: você não vai às prateleiras sozinho — pede ao gerente, que sabe onde cada documento está, garante que dois funcionários não peguem o mesmo documento ao mesmo tempo e controla quem tem permissão de acessar o quê.

2.2 Arquitetura de Três Camadas (ANSI/SPARC)



A **camada externa** é o que cada usuário enxerga. A **camada conceitual** descreve a estrutura lógica do banco. A **camada interna** descreve como os dados são fisicamente armazenados. Essa separação garante a **independência de dados**.

2.3 Principais SGBDs do Mercado

- **MySQL:** open source, mais popular do mundo para aplicações web. Usado pelo Facebook (nos primeiros anos), YouTube, WordPress.
- **PostgreSQL:** open source, extremamente robusto, reconhecido pela conformidade com padrões SQL e suporte a tipos avançados.
- **Oracle Database:** o mais utilizado em grandes corporações, com recursos avançados de alta disponibilidade e segurança.
- **Microsoft SQL Server:** integrado ao ecossistema Microsoft, amplamente usado com .NET e Power BI.
- **SQLite:** banco embutido em arquivo único, usado em aplicativos mobile e sistemas embarcados.
- **MongoDB:** NoSQL orientado a documentos, armazena dados em BSON (JSON binário).
- **Redis:** banco em memória chave-valor, usado para cache, sessões e filas.

2.4 Instalação do MySQL e MySQL Workbench

Windows: baixe o MySQL Installer em mysql.com, escolha "Developer Default", defina a senha do root e conclua a instalação.

Ubuntu Linux:

```
sudo apt update  
sudo apt install mysql-server  
sudo mysql_secure_installation
```

Primeiros comandos:

```
SELECT VERSION();  
SHOW DATABASES;
```

```
CREATE DATABASE meu_primeiro_banco CHARACTER SET utf8mb4;  
USE meu_primeiro_banco;  
SELECT DATABASE();
```

2.5 Exercícios do Capítulo 2

Exercício 1 — Desenhe a arquitetura de três camadas de um sistema hospitalar. Descreva o que estaria em cada camada para os perfis de médico, paciente, recepcionista e financeiro.

Exercício 2 — Para cada cenário abaixo, indique o SGBD mais adequado e justifique:

- a) Um blog pessoal com orçamento zero
- b) O sistema de transações de um grande banco
- c) Um app mobile de lista de tarefas
- d) Uma plataforma de recomendações baseadas em grafos) Monitoramento de 50.000 sensores registrando leituras por segundo

Exercício 3 — Instale o MySQL e o Workbench, crie um banco "exercicio_cap2", uma tabela "produto" com 5 colunas, insira 3 registros e documente com prints.

CAPÍTULO 3 — MODELAGEM CONCEITUAL E DIAGRAMA ENTIDADE-RELACIONAMENTO

3.1 Por que Modelar antes de Implementar

Imagine construir uma casa sem planta arquitetônica. Com bancos de dados é o mesmo: a modelagem é a "planta" do banco. Feita antes de qualquer SQL, ela previne problemas difíceis e caros de corrigir em produção.

A modelagem passa por três fases: **conceitual** (o quê), **lógica** (como se relacionam) e **física** (como implementar no SGBD).

3.2 Entidades, Atributos e Relacionamentos

Uma **entidade** representa um objeto do mundo real sobre o qual se quer armazenar informações. Exemplos: Cliente, Produto, Pedido, Funcionário.

Um **atributo** é uma propriedade que descreve uma entidade. Cliente tem: nome, CPF, e-mail, telefone, data de nascimento.

Um **relacionamento** descreve como duas entidades se associam. Um cliente *realiza* pedidos. Um funcionário *pertence* a um departamento.

3.3 Tipos de Atributos

Tipo	Descrição	Exemplo	Símbolo DER
Simples	Indivisível, valor único	CPF, preço	Elipse simples
Composto	Formado por subatributos	Endereço (rua, número, bairro...)	Elipse com ramificações
Multivalorado	Múltiplos valores possíveis	Telefones do cliente	Elipse dupla
Derivado	Calculado a partir de outros	Idade (da data de nascimento)	Elipse tracejada
Identificador	Identifica unicamente cada instância	CPF, matrícula	Nome sublinhado

3.4 Cardinalidade

A cardinalidade define quantas instâncias de uma entidade se relacionam com quantas da outra.

- **1:1 (Um para Um):** uma pessoa tem no máximo uma CNH, e uma CNH pertence a no máximo uma pessoa.
- **1:N (Um para Muitos):** um cliente pode ter muitos pedidos, mas cada pedido pertence a apenas um cliente. É o tipo mais comum.
- **N:M (Muitos para Muitos):** um pedido pode conter vários produtos, e um produto pode aparecer em vários pedidos.
- **Participação obrigatória (total):** todo pedido DEVE ter um cliente. **Participação opcional (parcial):** um cliente PODE ter pedidos (ou não).

3.5 Entidade Fraca

Uma entidade fraca não pode ser identificada unicamente por seus próprios atributos — depende de outra entidade (entidade forte) para sua identificação completa.

Exemplo: Item de Pedido. O item nº 1 só faz sentido dentro de um pedido específico. Sem o pedido, o item não existe. É identificado pela combinação (*idpedido* + *numitem*).

3.6 Generalização e Especialização

Quando diferentes entidades compartilham atributos comuns e também têm atributos específicos, usamos generalização e especialização (equivalente à herança na POO).

```
[PESSOA]
  nome, cpf, email
/      \
[FUNCIONÁRIO]  [CLIENTE]
matricula,      limite_credito,
cargo, salario  data_cadastro
```

3.7 Notações do DER

Notação Peter Chen: retângulos para entidades, elipses para atributos, losangos para relacionamentos. Mais expressiva e detalhada, usada para fins didáticos.

Notação Crow's Foot: mais compacta, usada em ferramentas profissionais como MySQL Workbench e Lucidchart. Cardinalidade representada na ponta das linhas.

Crow's Foot — Cardinalidade:

```
—|—  Exatamente um
—○—  Zero ou um
—|—< Um ou muitos
—○—< Zero ou muitos
```

3.8 DER Completo: Sistema de Clínica Médica

Entidades identificadas: Paciente, Médico, Especialidade, Consulta, Exame, Laboratório.

Relacionamentos:

- Paciente REALIZA Consulta: 1:N
- Médico ATENDE Consulta: 1:N
- Médico POSSUI Especialidade: N:M
- Consulta SOLICITA Exame: 1:N (exame é entidade fraca)
- Exame REALIZADO_POR Laboratório: N:1

3.9 Exercícios do Capítulo 3

Exercício 1 — Para cada par de entidades, defina a cardinalidade e explique:

- a) Pessoa — Passaporte
- b) Aluno — Disciplina
- c) Funcionário — Departamento
- d) Livro — Autore) Produto — Categoria

- **Exercício 2** — Construa o DER completo no brModelo para uma biblioteca universitária com: livros, autores, usuários (alunos e professores), empréstimos, categorias, multas por atraso.
- **Exercício 3** — Identifique e classifique os atributos da entidade Funcionário: *nomecompleto*, *CPF*, *datanascimento*, *idade*, *salariobruto*, *salarioliquido*, *endereço completo*, *telefonesdecontato*, *cargo*, *dataadmissao*, *tempodeempresa*, *departamento*.

CAPÍTULO 4 — MODELO LÓGICO E MODELO FÍSICO

4.1 Regras de Transformação do DER para o Modelo Lógico

- **Regra 1 — Entidade forte → Tabela.** Atributos viram colunas. Identificador vira PK.
- **Regra 2 — Relacionamento 1:N.** PK do lado "1" vai como FK para o lado "N".
- **Regra 3 — Relacionamento N:M.** Cria-se tabela associativa com as PKs de ambas as entidades como FKs (formam a PK composta da nova tabela).
- **Regra 4 — Relacionamento 1:1.** PK de uma entidade vai como FK na outra (geralmente na de participação obrigatória).
- **Regra 5 — Atributo composto.** Cada subatributo vira uma coluna separada. O atributo composto em si não gera coluna.
- **Regra 6 — Atributo multivalorado.** Cria-se nova tabela com a PK da entidade como FK e o atributo como coluna.
- **Regra 7 — Entidade fraca.** Vira tabela com a PK da entidade forte como FK, combinada com o identificador parcial para formar a PK composta.

4.2 Tipos de Dados no MySQL

Tipos Inteiros

Tipo	Bytes	Faixa	Uso Recomendado
TINYINT	1	-128 a 127	Flags, idades, notas 0-10
SMALLINT	2	-32.768 a 32.767	Ano letivo, códigos pequenos
INT	4	±2,1 bilhões	IDs da maioria das tabelas
BIGINT	8	±9,2 quintilhões	IDs de tabelas de grande porte

Tipos Decimais

Tipo	Uso
DECIMAL(p,s)	Sempre use para dinheiro. Armazenamento exato. Ex: DECIMAL(10,2)
FLOAT / DOUBLE	Medições científicas. Nunca use para valores monetários.

Tipos de Texto

Tipo	Limite	Uso
CHAR(n)	255 caracteres (fixo)	CPF, CEP, siglas (tamanho sempre igual)
VARCHAR(n)	65.535 (variável)	Nomes, e-mails, descrições curtas
TEXT	64 KB	Descrições longas, observações
ENUM(...)	Lista predefinida	Status, tipo de contato

Tipos de Data

Tipo	Formato	Uso
DATE	YYYY-MM-DD	Data de nascimento, data do pedido
TIME	HH:MM:SS	Horário de aula, duração
DATETIME	YYYY-MM-DD HH:MM:SS	Criação/atualização de registros
TIMESTAMP	YYYY-MM-DD HH:MM:SS	Controle automático com DEFAULT CURRENT_TIMESTAMP
YEAR	YYYY	Ano letivo, ano de fabricação

4.3 Constraints

Constraint	Descrição
PRIMARY KEY	Identifica unicamente cada linha. Não permite NULL nem duplicata.
FOREIGN KEY	Referencia a PK de outra tabela. Garante integridade referencial.
UNIQUE	Garante unicidade. Permite um único NULL.
NOT NULL	Proíbe valores nulos. Use para campos obrigatórios.
DEFAULT	Define valor padrão quando não especificado no INSERT.
CHECK	Valida condição sobre o valor inserido (MySQL 8.0.16+).
AUTO_INCREMENT	Gera inteiro crescente automaticamente.

Opções de FK:

- ON DELETE RESTRICT — bloqueia exclusão do pai se houver filhos (padrão)
- ON DELETE CASCADE — exclui os filhos automaticamente
- ON DELETE SET NULL — define NULL nas FKs filhas

4.4 Script DDL Completo: Clínica Saúde Plena

```
CREATE DATABASE IF NOT EXISTS clinica_saude_plena
    CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

USE clinica_saude_plena;

CREATE TABLE especialidade (
    id_especialidade INT NOT NULL AUTO_INCREMENT,
    nome            VARCHAR(80) NOT NULL,
    descricao       TEXT,
    ativa           BOOLEAN NOT NULL DEFAULT TRUE,
    CONSTRAINT pk_especialidade PRIMARY KEY (id_especialidade),
    CONSTRAINT uq_especialidade_nome UNIQUE (nome)
);

CREATE TABLE medico (
    id_medico INT NOT NULL AUTO_INCREMENT,
    crm        VARCHAR(12) NOT NULL,
    nome       VARCHAR(100) NOT NULL,
    email      VARCHAR(100),
    ativo      BOOLEAN NOT NULL DEFAULT TRUE,
    CONSTRAINT pk_medico PRIMARY KEY (id_medico),
    CONSTRAINT uq_medico_crm UNIQUE (crm)
);

CREATE TABLE paciente (
    id_paciente INT NOT NULL AUTO_INCREMENT,
    nome        VARCHAR(100) NOT NULL,
    cpf         CHAR(11) NOT NULL,
    data_nascimento DATE NOT NULL,
    email       VARCHAR(100),
    logradouro  VARCHAR(150),
```

```
numero_end    VARCHAR(10),
bairro        VARCHAR(80),
cidade        VARCHAR(80),
estado        CHAR(2),
cep           CHAR(8),
ativo         BOOLEAN    NOT NULL DEFAULT TRUE,
data_cadastro DATETIME    NOT NULL DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT pk_paciente PRIMARY KEY (id_paciente),
CONSTRAINT uq_paciente_cpf UNIQUE (cpf),
CONSTRAINT chk_paciente_cpf CHECK (cpf REGEXP '[0-9]{11}$')
);
```

```
CREATE TABLE consulta (
    id_consulta    INT     NOT NULL AUTO_INCREMENT,
    id_paciente    INT     NOT NULL,
    id_medico      INT     NOT NULL,
    data_consulta  DATE    NOT NULL,
    horario        TIME    NOT NULL,
    duracao_prevista SMALLINT NOT NULL DEFAULT 30,
    diagnostico    TEXT,
    status         ENUM('AGENDADA','REALIZADA','CANCELADA','FALTA')
                  NOT NULL DEFAULT 'AGENDADA',
    CONSTRAINT pk_consulta PRIMARY KEY (id_consulta),
    CONSTRAINT fk_cons_paciente FOREIGN KEY (id_paciente)
        REFERENCES paciente(id_paciente) ON DELETE RESTRICT,
    CONSTRAINT fk_cons_medico FOREIGN KEY (id_medico)
        REFERENCES medico(id_medico) ON DELETE RESTRICT
);
```

```
CREATE INDEX idx_consulta_data    ON consulta(data_consulta);
CREATE INDEX idx_consulta_paciente ON consulta(id_paciente);
CREATE INDEX idx_consulta_medico  ON consulta(id_medico);
```

4.5 Exercícios do Capítulo 4

Exercício 1 — Transforme o DER da biblioteca (Capítulo 3) em modelo lógico, documentando cada decisão com justificativa.

Exercício 2 — Para cada situação, justifique o tipo de dado mais adequado:

- a) Número do CPF brasileiro
- b) Valor de transação financeira até R\$ 999.999,99
- c) Status de pedido (Pendente, Confirmado, Enviado, Entregue, Cancelado)
- d) Senha criptografada em SHA-256 (sempre 64 caracteres hex)
- e) Data de nascimento de um cliente

Exercício 3 — O script abaixo tem 8 erros de projeto. Identifique cada um e escreva a versão corrigida:

```
CREATE TABLE venda (  
  codigo    VARCHAR(10),  
  valor     FLOAT,  
  data_venda VARCHAR(20),  
  cliente   VARCHAR(100),  
  desconto  INT,  
  pago      VARCHAR(5),  
  observacao CHAR(2000)  
);
```

CAPÍTULO 5 — NORMALIZAÇÃO DE DADOS

5.1 O Problema da Redundância

Tabela desnormalizada de exemplo:

num_pedido	nome_aluno	cod_disciplina	nome_disciplina	ch	cpf_prof	nome_prof	bimestre	nota
2024001	Ana Lima	BD01	Banco de Dados	80h	9999...	Prof. Carlos	1	8.5
2024001	Ana Lima	BD01	Banco de Dados	80h	9999...	Prof. Carlos	2	9.0

Anomalias identificadas:

- **Anomalia de inserção:** não é possível cadastrar um professor sem que ele já tenha uma nota lançada.
- **Anomalia de atualização:** se o nome do professor mudar, todas as linhas precisam ser atualizadas.
- **Anomalia de exclusão:** excluir a única nota de uma disciplina apaga os dados da própria disciplina.

5.2 Dependência Funcional

Dizemos que B é funcionalmente dependente de A quando, para cada valor de A, existe um único valor de B correspondente.

Notação: $A \rightarrow B$ ("A determina B")

Tipos:

- **Total:** B depende de toda a chave composta
- **Parcial:** B depende de apenas parte da chave composta (viola 2FN)
- **Transitiva:** B depende de A indiretamente através de C (viola 3FN)

5.3 Primeira Forma Normal — 1FN

Uma tabela está na 1FN quando: todos os atributos são atômicos (indivisíveis), não existem grupos repetitivos e existe chave primária definida.

Violação: campo "telefones" contendo "(41)99999-1111, (41)88888-2222" em uma única coluna.

Correção: criar tabela separada *TELEFONECLIENTE* (*idcliente*, telefone, tipo).

5.4 Segunda Forma Normal — 2FN

Uma tabela está na 2FN quando está na 1FN e todos os atributos não-chave dependem funcionalmente de **toda** a chave primária (não apenas de parte dela). Aplica-se somente a tabelas com chave composta.

Violação exemplo:

Tabela NOTA com PK (*nummatricula*, *coddisciplina*, bimestre):

- *nomealuno* depende só de *nummatricula* → dependência parcial
- *nomedisciplina* depende só de *coddisciplina* → dependência parcial

Correção: separar ALUNO e DISCIPLINA em tabelas próprias.

5.5 Terceira Forma Normal — 3FN

Uma tabela está na 3FN quando está na 2FN e não existem dependências transitivas — nenhum atributo não-chave depende de outro atributo não-chave.

Violação exemplo:

- Tabela ALUNO: *nummatricula* → *codcurso* → *nome_curso*
- *nomecurso* depende transitivamente de *nummatricula* (via *cod_curso*).

Correção: separar CURSO em tabela própria.

5.6 Processo Completo de Normalização — Exemplo

- **Tabela original (0FN):** PEDIDO_COMPLETO com todos os dados em uma linha, incluindo grupos repetitivos de produtos.
- **Após 1FN:** eliminar grupos repetitivos (produto1, produto2...) — cada combinação pedido+produto vira uma linha. Criar tabela para telefones.
- **Após 2FN:** separar atributos com dependência parcial — criar tabelas CLIENTE e PRODUTO independentes.
- **Após 3FN:** separar dependências transitivas — criar tabelas VENDEDOR e DEPARTAMENTO.

Resultado final:

- CLIENTE (*codcli*, *nomecli*, *cidade_cli*)
- VENDEDOR (*codvend*, *nomevend*, *id_departamento*)
- DEPARTAMENTO (*iddepartamento*, *nomedepartamento*)
- PEDIDO (*numpedido*, *data*, *codcli*, *cod_vend*)
- PRODUTO (*codproduto*, *nomeproduto*)
- ITEMPEDIDO (*numpedido*, *cod_produto*, *qtd*, *preco*)

5.7 Desnormalização Estratégica

A desnormalização é o processo intencional de introduzir redundância controlada para ganhar performance de leitura. É uma decisão técnica que deve ser tomada após análise cuidadosa de gargalos mensuráveis.

Regra de ouro: normalize primeiro, desnormalize depois somente quando houver evidência mensurável de problema de performance.

5.8 Exercícios do Capítulo 5

Exercício 1 — Normalize a tabela abaixo até a 3FN, documentando cada passo:

id_aluno	nome_aluno	cpf	id_curso	nome_curso	ch_curso	id_professor	nome_professor	sala	nota
1	Ana Lima	111...	BD	Banco de Dados	80h	5	Prof. Carlos	201	8.5

Exercício 2 — Para cada dependência funcional, classifique como total, parcial ou transitiva:

a) Chave (*idpedido*, *idproduto*) → *idproduto* → *nome*

produto) b) Chave (*idpedido*, *id*

produto) → *quantidade* c) Chave *idfuncionario* → *iddepartamento* → *nome_departamento*

Exercício 3 — Cite dois exemplos reais onde a desnormalização seria tecnicamente justificável, descrevendo o dado duplicado, o motivo e o ganho esperado.

CAPÍTULO 6 — LINGUAGEM SQL: DDL E DML

6.1 Categorias do SQL

Categoria	Significado	Comandos
DDL	Data Definition Language	CREATE, ALTER, DROP, TRUNCATE
DML	Data Manipulation Language	INSERT, UPDATE, DELETE
DQL	Data Query Language	SELECT
DCL	Data Control Language	GRANT, REVOKE
TCL	Transaction Control Language	BEGIN, COMMIT, ROLLBACK

SQL é uma linguagem **declarativa**: você declara o que quer, não como obter. O SGBD decide internamente a melhor estratégia de execução.

6.2 DDL — Criando Estruturas

-- Criar banco de dados

```
CREATE DATABASE sistema_loja  
    CHARACTER SET utf8mb4  
    COLLATE utf8mb4_unicode_ci;
```

USE sistema_loja;

-- Criar tabela completa

```
CREATE TABLE produto (  
    id_produto    INT          NOT NULL AUTO_INCREMENT,  
    codigo        VARCHAR(20)  NOT NULL,  
    nome          VARCHAR(100) NOT NULL,  
    descricao     TEXT,  
    preco         DECIMAL(10,2) NOT NULL CHECK (preco >= 0),  
    estoque       INT          NOT NULL DEFAULT 0,  
    id_categoria  INT          NOT NULL,  
    ativo         BOOLEAN      NOT NULL DEFAULT TRUE,  
    data_cadastro DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    CONSTRAINT pk_produto      PRIMARY KEY (id_produto),  
    CONSTRAINT uq_produto_codigo UNIQUE (codigo),  
    CONSTRAINT fk_prod_categoria FOREIGN KEY (id_categoria)  
        REFERENCES categoria(id_categoria)  
        ON DELETE RESTRICT ON UPDATE CASCADE  
);
```

-- Modificar tabela

```
ALTER TABLE produto ADD COLUMN peso DECIMAL(8,3);  
ALTER TABLE produto MODIFY COLUMN nome VARCHAR(150) NOT NULL;  
ALTER TABLE produto DROP COLUMN peso;
```

-- Índices

```
CREATE INDEX idx_produto_nome ON produto(nome);
CREATE INDEX idx_prod_cat_preco ON produto(id_categoria, preco);

-- Excluir tabela
DROP TABLE IF EXISTS produto;

-- Limpar tabela (mantém estrutura)
TRUNCATE TABLE produto;
```

6.3 DML — Manipulando Dados

```
-- INSERT: inserção simples (especifique sempre as colunas)
INSERT INTO produto (codigo, nome, preco, id_categoria)
VALUES ('PROD001', 'Notebook Dell', 3299.99, 1);

-- INSERT múltiplo (mais eficiente)
INSERT INTO produto (codigo, nome, preco, id_categoria) VALUES
    ('PROD002', 'Mouse Logitech', 89.90, 1),
    ('PROD003', 'Teclado Mecânico', 350.00, 1),
    ('PROD004', 'Monitor 24 pol', 1200.00, 1);

-- UPDATE: sempre use WHERE!
UPDATE produto
SET preco = 2999.99,
    data_atualizacao = NOW()
WHERE codigo = 'PROD001';

-- UPDATE com cálculo
UPDATE produto
SET preco = preco * 1.10 -- reajuste de 10%
WHERE id_categoria = 1;

-- DELETE: sempre use WHERE!
```

```
DELETE FROM produto WHERE id_produto = 10;
```

```
-- Antes de DELETE, confirme a quantidade:
```

```
SELECT COUNT(*) FROM produto WHERE ativo = FALSE;
```

```
DELETE FROM produto WHERE ativo = FALSE;
```

6.4 Transações — ACID na Prática

```
-- Transferência bancária: o exemplo clássico
```

```
START TRANSACTION;
```

```
UPDATE conta
```

```
SET saldo = saldo - 1000.00
```

```
WHERE id_conta = 101;
```

```
UPDATE conta
```

```
SET saldo = saldo + 1000.00
```

```
WHERE id_conta = 205;
```

```
INSERT INTO extrato (id_conta_origem, id_conta_destino, valor)
```

```
VALUES (101, 205, 1000.00);
```

```
COMMIT; -- confirma tudo
```

```
-- Se qualquer erro ocorrer:
```

```
-- ROLLBACK; -- desfaz tudo
```

Propriedades ACID:

- **Atomicidade** — tudo ou nada
- **Consistência** — estado válido antes e depois
- **Isolamento** — transações não interferem entre si
- **Durabilidade** — dados confirmados persistem mesmo após falhas

6.5 Exercícios do Capítulo 6

- **Exercício 1** — Crie o banco "sistemaloja" com as tabelas: *categoria*, *fornecedor*, *produto*, *cliente*, *pedido* e *itempedido*, com todos os tipos de dados, constraints e índices adequados.
- **Exercício 2** — Popule o banco com dados realistas: 5 categorias, 3 fornecedores, 10 produtos, 5 clientes, 3 pedidos com 2 a 4 itens cada.
- **Exercício 3** — Escreva os comandos para:

- a) Aumentar o preço de todos os produtos de "Informática" em 8%
- b) Desativar clientes que não comprem há mais de 2 anos
- c) Excluir pedidos cancelados com mais de 1 ano
- d) Adicionar coluna "peso_kg DECIMAL(6,2)" à tabela produtoe) Criar índice composto para buscas de pedidos por cliente e data

CAPÍTULO 7 — CONSULTAS AVANÇADAS EM SQL

7.1 Estrutura Completa do SELECT

```
SELECT [DISTINCT] lista_de_colunas
FROM   tabela_principal
[JOIN  outras_tabelas ON condição]
[WHERE condição_de_filtro]
[GROUP BY colunas_de_agrupamento]
[HAVING condição_sobre_grupos]
[ORDER BY colunas [ASC|DESC]]
[LIMIT quantidade [OFFSET deslocamento]];
```

Ordem de execução lógica: FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT.

7.2 Filtros e Funções Essenciais

-- Filtros com WHERE

```
SELECT * FROM produto WHERE preco BETWEEN 100 AND 500;
```

```
SELECT * FROM cliente WHERE nome LIKE 'Ana%';
```

```
SELECT * FROM pedido WHERE status IN ('PENDENTE', 'CONFIRMADO');
```

```
SELECT * FROM produto WHERE id_fornecedor IS NULL;
```

-- Funções de string

```
SELECT CONCAT(nome, ' — ', cpf) FROM cliente;
```

```
SELECT UPPER(nome), LOWER(email) FROM cliente;
```

```
SELECT TIMESTAMPDIFF(YEAR, data_nascimento, CURDATE()) AS idade FROM cliente;
```

-- Funções de data

```
SELECT DATE_FORMAT(data_pedido, '%d/%m/%Y') FROM pedido;
```

```
SELECT * FROM pedido WHERE data_pedido >= DATE_SUB(CURDATE(), INTERVAL 30 DAY);
```

-- Agregação com GROUP BY e HAVING

```
SELECT
```

```
    d.nome AS departamento,
```

```
    COUNT(f.id_funcionario) AS qtd_funcionarios,
```

```
    AVG(f.salario) AS salario_medio
```

```
FROM funcionario f
```

```
JOIN departamento d ON f.id_departamento = d.id_departamento
```

```
WHERE f.ativo = TRUE
```

```
GROUP BY d.id_departamento, d.nome
```

```
HAVING AVG(f.salario) > 3000
```

```
ORDER BY salario_medio DESC;
```

7.3 JOINS em Profundidade

-- INNER JOIN: apenas os que têm correspondência nos dois lados

```
SELECT p.id_pedido, c.nome AS cliente, p.valor_total  
FROM pedido p  
INNER JOIN cliente c ON p.id_cliente = c.id_cliente;
```

-- LEFT JOIN: todos da esquerda + correspondências da direita

-- Clientes que NUNCA compraram:

```
SELECT c.id_cliente, c.nome  
FROM cliente c  
LEFT JOIN pedido p ON c.id_cliente = p.id_cliente  
WHERE p.id_pedido IS NULL;
```

-- SELF JOIN: hierarquia de funcionários

```
SELECT f.nome AS funcionario, g.nome AS gerente  
FROM funcionario f  
LEFT JOIN funcionario g ON f.id_gerente = g.id_funcionario;
```

Diagrama visual dos JOINS:

INNER JOIN	LEFT JOIN
$[A] \cap [B]$	$[A] \cup (A \cap B)$
só a interseção	todo A + interseção

RIGHT JOIN	FULL OUTER (UNION)
$(A \cap B) \cup [B]$	$[A] \cup [B]$
todo B + interseção	tudo

7.4 Subqueries e CTEs

-- Subquery no WHERE: produtos acima da média de preço

```
SELECT nome, preco FROM produto
```

```
WHERE preco > (SELECT AVG(preco) FROM produto);
```

-- EXISTS: clientes com pedido pendente

```
SELECT c.nome FROM cliente c
```

```
WHERE EXISTS (
```

```
    SELECT 1 FROM pedido p
```

```
    WHERE p.id_cliente = c.id_cliente
```

```
    AND p.status = 'PENDENTE'
```

```
);
```

-- CTE com WITH: vendas do semestre + ranking

```
WITH vendas_sem AS (
```

```
    SELECT c.id_cliente, c.nome,
```

```
           SUM(p.valor_total) AS total
```

```
    FROM cliente c
```

```
    JOIN pedido p ON c.id_cliente = p.id_cliente
```

```
    WHERE p.data_pedido >= DATE_SUB(CURDATE(), INTERVAL 6 MONTH)
```

```
    GROUP BY c.id_cliente, c.nome
```

```
)
```

```
SELECT *, RANK() OVER (ORDER BY total DESC) AS posicao
```

```
FROM vendas_sem
```

```
ORDER BY posicao;
```

7.5 Window Functions

```
-- ROW_NUMBER: numerar linhas por departamento
SELECT nome, salario,
       ROW_NUMBER() OVER (
         PARTITION BY id_departamento
         ORDER BY salario DESC
       ) AS posicao
FROM funcionario;

-- RANK e DENSE_RANK: ranking com e sem lacunas
SELECT nome, salario,
       RANK() OVER (ORDER BY salario DESC) AS rank_lacuna,
       DENSE_RANK() OVER (ORDER BY salario DESC) AS rank_sem_lacuna
FROM funcionario;

-- LAG: variação em relação ao mês anterior
SELECT
  YEAR(data_pedido) AS ano,
  MONTH(data_pedido) AS mes,
  SUM(valor_total) AS faturamento,
  SUM(valor_total) - LAG(SUM(valor_total))
    OVER (ORDER BY YEAR(data_pedido), MONTH(data_pedido))
    AS variacao
FROM pedido
GROUP BY YEAR(data_pedido), MONTH(data_pedido);
```

7.6 Exercícios do Capítulo 7

Exercício 1 — Escreva as queries para:

- a) Listar todos os produtos com nome da categoria, incluindo produtos sem categoria
- b) Top 5 produtos mais vendidos no último trimestre
- c) Faturamento mensal por vendedor no ano atual
- d) Clientes que compraram na última semana mas nunca tinham comprado antes)
- e) Ticket médio por categoria, somente categorias acima de R\$ 200

Exercício 2 — Crie um relatório de clientes inativos em risco: que fizeram ao menos 3 pedidos mas não compram há mais de 90 dias. Mostre nome, e-mail, data da última compra e valor total histórico.

Exercício 3 — Explique a diferença entre WHERE e HAVING com um exemplo onde usar WHERE no lugar de HAVING produziria resultado incorreto.

CAPÍTULO 8 — OBJETOS DE BANCO DE DADOS

8.1 Views

```
-- Criar view gerencial
CREATE OR REPLACE VIEW vw_pedidos_detalhados AS
SELECT
    p.id_pedido, p.data_pedido, p.status, p.valor_total,
    c.nome AS cliente, c.email,
    f.nome AS vendedor,
    d.nome AS departamento
FROM pedido p
JOIN cliente c    ON p.id_cliente    = c.id_cliente
JOIN funcionario f ON p.id_funcionario = f.id_funcionario
JOIN departamento d ON f.id_departamento = d.id_departamento;

-- Usar a view como tabela
SELECT * FROM vw_pedidos_detalhados
WHERE status = 'PENDENTE'
ORDER BY data_pedido;

-- View de catálogo público (oculta dados sensíveis)
CREATE OR REPLACE VIEW vw_catalogo_ecommerce AS
SELECT
    p.id_produto, p.nome, p.descricao, p.preco,
```

```
c.nome AS categoria,
CASE
    WHEN p.estoque > 10 THEN 'Disponível'
    WHEN p.estoque > 0 THEN 'Últimas unidades'
    ELSE 'Indisponível'
END AS disponibilidade
FROM produto p
JOIN categoria c ON p.id_categoria = c.id_categoria
WHERE p.ativo = TRUE;
```

8.2 Stored Procedures

```
DELIMITER //

CREATE PROCEDURE sp_registrar_pedido(
    IN p_id_cliente    INT,
    IN p_id_funcionario INT,
    IN p_id_produto    INT,
    IN p_quantidade    INT,
    OUT p_resultado    VARCHAR(200),
    OUT p_id_pedido    INT
)
BEGIN
    DECLARE v_estoque INT;
    DECLARE v_preco   DECIMAL(10,2);

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SET p_resultado = 'ERRO: Transação revertida.';
        SET p_id_pedido = -1;
    END;
```

```
SELECT estoque, preco INTO v_estoque, v_preco
FROM produto WHERE id_produto = p_id_produto FOR UPDATE;

IF v_estoque < p_quantidade THEN
    SET p_resultado = CONCAT('ERRO: Estoque insuficiente. Disponível: ',
v_estoque);
    SET p_id_pedido = -1;
ELSE
    START TRANSACTION;

    INSERT INTO pedido (id_cliente, id_funcionario, data_pedido, valor_total)
    VALUES (p_id_cliente, p_id_funcionario, NOW(), v_preco * p_quantidade);

    SET p_id_pedido = LAST_INSERT_ID();

    INSERT INTO item_pedido (id_pedido, id_produto, quantidade, preco_unitario)
    VALUES (p_id_pedido, p_id_produto, p_quantidade, v_preco);

    UPDATE produto
    SET estoque = estoque - p_quantidade
    WHERE id_produto = p_id_produto;

    COMMIT;
    SET p_resultado = CONCAT('SUCESSO: Pedido #', p_id_pedido, ' registrado.');
```

END IF;

END //

DELIMITER ;

-- Chamar a procedure

CALL sp_registrar_pedido(1, 3, 5, 2, @resultado, @id_pedido);

SELECT @resultado, @id_pedido;

8.3 Functions

```
DELIMITER //
```

```
-- Function: calcular desconto progressivo
```

```
CREATE FUNCTION fn_desconto(p_valor DECIMAL(10,2))
```

```
RETURNS DECIMAL(10,2)
```

```
DETERMINISTIC NO SQL
```

```
BEGIN
```

```
    DECLARE v_desc DECIMAL(10,2);
```

```
    CASE
```

```
        WHEN p_valor < 100 THEN SET v_desc = 0;
```

```
        WHEN p_valor < 500 THEN SET v_desc = p_valor * 0.05;
```

```
        WHEN p_valor < 1000 THEN SET v_desc = p_valor * 0.10;
```

```
        ELSE          SET v_desc = p_valor * 0.15;
```

```
    END CASE;
```

```
    RETURN ROUND(v_desc, 2);
```

```
END //
```

```
-- Function: formatar CPF
```

```
CREATE FUNCTION fn_cpf(p CHAR(11))
```

```
RETURNS VARCHAR(14)
```

```
DETERMINISTIC NO SQL
```

```
BEGIN
```

```
    RETURN CONCAT(
```

```
        SUBSTRING(p,1,3),',',SUBSTRING(p,4,3),',',
```

```
        SUBSTRING(p,7,3),',',SUBSTRING(p,10,2)
```

```
    );
```

```
END //
```

```
DELIMITER ;
```

```
-- Uso em query
```

```
SELECT nome, fn_cpf(cpf) AS cpf_formatado,  
       valor_total, fn_desconto(valor_total) AS desconto  
FROM pedido p JOIN cliente c ON p.id_cliente = c.id_cliente;
```

8.4 Triggers

```
DELIMITER //
```



```
-- Trigger de auditoria de preço  
CREATE TRIGGER trg_auditoria_preco  
AFTER UPDATE ON produto  
FOR EACH ROW  
BEGIN  
    IF OLD.preco <> NEW.preco THEN  
        INSERT INTO auditoria_preco (  
            id_produto, preco_anterior, preco_novo,  
            variacao_perc, usuario, data_alteracao  
        ) VALUES (  
            NEW.id_produto, OLD.preco, NEW.preco,  
            ROUND((NEW.preco - OLD.preco) / OLD.preco * 100, 2),  
            USER(), NOW()  
        );  
    END IF;  
END //
```



```
-- Trigger de proteção: impedir exclusão de cliente com pedidos ativos  
CREATE TRIGGER trg_proteger_cliente  
BEFORE DELETE ON cliente  
FOR EACH ROW  
BEGIN  
    DECLARE v_ativos INT;  
    SELECT COUNT(*) INTO v_ativos FROM pedido  
    WHERE id_cliente = OLD.id_cliente
```

```
AND status IN ('PENDENTE','CONFIRMADO','ENVIADO');

IF v_ativos > 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Cliente tem pedidos ativos. Exclusão bloqueada.';
END IF;
END //

DELIMITER ;
```

Disponibilidade de OLD e NEW:

Evento	OLD	NEW
INSERT	Não	Sim (valores inseridos)
UPDATE	Sim (antes)	Sim (depois)
DELETE	Sim (valores excluídos)	Não

8.5 Exercícios do Capítulo 8

Exercício 1 — Crie as views para o sistema *loja*: *vwestoquecritico*, *vrankingvendedores*, *vwclientespremium*, *vwprodutossemvenda*.

Exercício 2 — Crie as procedures: *spcadastrarcliente* (com validação de CPF e e-mail), *spcancelarpedido* (com restauração de estoque), *sprelatoriovendas_periodo*.

Exercício 3 — Implemente sistema de auditoria completo com triggers para: alterações de dados de cliente, exclusões de produto e mudanças de status de pedido.

CAPÍTULO 9 — SEGURANÇA, BACKUP E ADMINISTRAÇÃO

9.1 Principais Ameaças

- **SQL Injection:** inserção de código malicioso em campos de entrada
- **Acesso não autorizado:** senhas fracas, permissões excessivas
- **Exposição de dados sensíveis:** dados armazenados sem criptografia
- **Ataques de força bruta:** tentativas sistemáticas de combinações de senha

9.2 SQL Injection — Ataque e Defesa

```
-- Sistema VULNERÁVEL (concatena input direto na query)
-- Login: admin'--   Senha: qualquer
-- Query gerada: SELECT * FROM usuario WHERE login = 'admin'-- ...
-- O "--" comenta o resto → entra SEM SABER A SENHA!

-- DEFESA: Stored Procedure com parâmetros
DELIMITER //
CREATE PROCEDURE sp_login_seguro(
    IN p_login    VARCHAR(50),
    IN p_senha    VARCHAR(100),
    OUT p_id_usuario INT,
    OUT p_perfil  VARCHAR(20)
)
BEGIN
    SELECT id_usuario, perfil
    INTO p_id_usuario, p_perfil
    FROM usuario
    WHERE login    = p_login
       AND senha_hash = SHA2(p_senha, 256)
       AND ativo    = TRUE;
```

```
END //
DELIMITER ;

-- DEFESA: Criptografia de dados sensíveis
SET @chave = 'ChaveSecreta2024!';
UPDATE cliente SET cpf_enc = AES_ENCRYPT(cpf, @chave);

-- Descriptografar
SELECT CAST(AES_DECRYPT(cpf_enc, @chave) AS CHAR) AS cpf FROM cliente;
```

9.3 Gerenciamento de Usuários e Privilégios

```
-- Criar usuários
CREATE USER 'dba_corp'@'localhost' IDENTIFIED BY 'Db@Corp2024!';
CREATE USER 'dev_corp'@'localhost' IDENTIFIED BY 'Dev@Corp2024!';
CREATE USER 'analista_corp'@'localhost' IDENTIFIED BY 'Ana@Corp2024!';
CREATE USER 'webapp_corp'@'localhost' IDENTIFIED BY 'Web@Corp2024!';

-- Conceder privilégios por perfil
GRANT ALL PRIVILEGES ON sistema_loja.* TO 'dba_corp'@'localhost';
GRANT SELECT, INSERT, UPDATE, DELETE,
      EXECUTE ON sistema_loja.* TO 'dev_corp'@'localhost';
GRANT SELECT ON sistema_loja.vw_resumo_vendas TO 'analista_corp'@'localhost';
GRANT SELECT ON sistema_loja.produto TO 'webapp_corp'@'localhost';
GRANT SELECT, INSERT ON sistema_loja.pedido TO 'webapp_corp'@'localhost';

FLUSH PRIVILEGES;

-- Verificar privilégios
SHOW GRANTS FOR 'dev_corp'@'localhost';

-- Revogar
REVOKE DROP, CREATE, ALTER ON sistema_loja.* FROM 'dev_corp'@'localhost';
```

9.4 LGPD e Banco de Dados

A Lei Geral de Proteção de Dados (LGPD — Lei 13.709/2018) impõe obrigações sobre o armazenamento de dados pessoais:

- **Minimização:** coletar apenas o necessário
- **Segurança:** proteção contra acesso não autorizado
- **Rastreabilidade:** registrar quem acessou ou alterou dados pessoais
- **Direito de exclusão:** o titular pode solicitar a exclusão de seus dados
- **Pseudonimização:** substituir identificadores diretos por códigos sem sentido

```
-- Anonimizar dados de cliente (para atender solicitação de exclusão LGPD)
```

```
UPDATE cliente SET
```

```
    nome    = CONCAT('ANONIMIZADO_', id_cliente),
```

```
    cpf      = CONCAT('000000000', LPAD(id_cliente, 2, '0')),
```

```
    email    = CONCAT('removido_', id_cliente, '@lgpd.anonimo'),
```

```
    telefone = NULL,
```

```
    ativo    = FALSE
```

```
WHERE id_cliente = 42;
```

9.5 Backup e Restauração

```
# Backup completo com opções recomendadas
```

```
mysqldump -u root -p \
```

```
    --single-transaction \
```

```
    --routines --triggers --events \
```

```
    --databases sistema_loja \
```

```
> backup_completo_$(date +%Y%m%d_%H%M%S).sql
```

```
# Backup comprimido
```

```
mysqldump -u root -p sistema_loja | gzip > backup.sql.gz
```

```
# Restaurar
```

```
mysql -u root -p sistema_loja < backup_completo.sql
```

```
# Restaurar comprimido
```

```
gunzip < backup.sql.gz | mysql -u root -p sistema_loja
```

Tipos de backup:

Tipo	Vantagem	Desvantagem
Completo	Restauração simples	Mais lento e maior
Incremental	Rápido e pequeno	Restauração complexa (precisa de todos os incrementais)
Diferencial	Equilíbrio entre os dois	Cresce com o tempo

9.6 Performance com EXPLAIN

```
-- Analisar plano de execução
```

```
EXPLAIN SELECT c.nome, COUNT(p.id_pedido)
```

```
FROM cliente c
```

```
JOIN pedido p ON c.id_cliente = p.id_cliente
```

```
WHERE p.data_pedido BETWEEN '2024-01-01' AND '2024-12-31'
```

```
GROUP BY c.id_cliente, c.nome;
```

```
-- Interpretando o EXPLAIN:
```

```
-- type = ALL → Full Table Scan (ruim)
```

```
-- type = range → faixa do índice (bom)
```

```
-- type = ref → índice com igualdade (muito bom)
```

```
-- rows → estimativa de linhas lidas (menor = melhor)
```

```
-- key → índice utilizado (NULL = sem índice)
```

```
-- Extra → "Using filesort" ou "Using temporary" = gargalos
```

```
-- Criar índices para otimizar
```

```
CREATE INDEX idx_pedido_data ON pedido(data_pedido);
```

```
CREATE INDEX idx_pedido_cliente ON pedido(id_cliente);
```

9.7 Exercícios do Capítulo 9

- **Exercício 1** — Crie uma política completa de segurança para o sistema_loja com os perfis: DBA, Gerente, Vendedor, Estoquista e Sistema Web. Escreva todos os comandos SQL.
- **Exercício 2** — Demonstre um SQL Injection em cenário de busca vulnerável e implemente a versão segura com procedure. Documente as diferenças.
- **Exercício 3** — Elabore um DRP (Plano de Recuperação de Desastres) contendo: tipos de backup, frequências, RTO, RPO e procedimento passo a passo.
- **Exercício 4** — Insira 100.000 registros, execute uma query sem índice, crie os índices adequados e compare com EXPLAIN. Documente a diferença de performance.

CAPÍTULO 10 — BIG DATA, NoSQL E QUALIDADE DE DADOS

V	Descrição	Exemplo
Volume	Quantidade massiva de dados	Petabytes de logs diários
Velocidade	Dados gerados e processados em alta velocidade	Sensores IoT a cada milissegundo
Variedade	Múltiplos formatos: estruturado, semiestruturado, não estruturado	Tabelas, JSON, imagens, vídeos